

Database After Graduation

Week 1

By: Narges Yarahmadi Gharaei

4 Feb 2026

About me

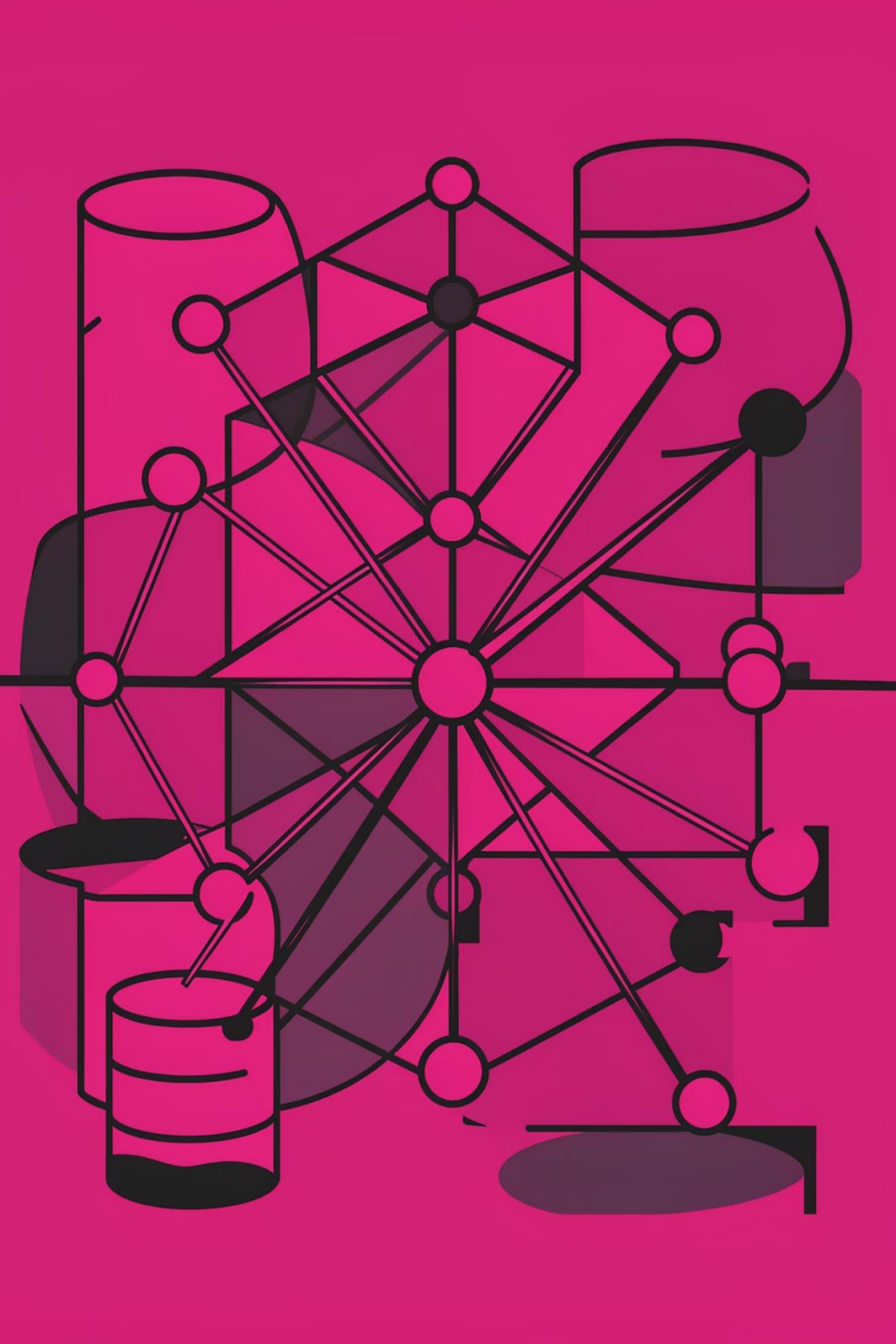
- PhD candidate in Computer Science
- TA experience in database courses
- Background in data systems and analytics (AI)
- Focus on hands-on, real-world learning



Narges Yarahmadi gharaei
PhD Candidate @ Dalhousie University



Section 1

An abstract graphic on the left side of the slide. It features several overlapping cylinders in various shades of blue and purple. A network of white nodes (circles) is connected by white lines, forming a complex web that spans across the cylinders. The background is a solid dark blue.

Before Querying: How to Think About Database Modeling

Before writing queries, we need to understand what we are querying. SQL is not just a syntax for fetching data. It is a language for asking questions about a **model of the real world**.

A database model is the bridge between:

- Real-world concepts (users, orders, payments)
- And structured data (tables, rows, keys)

If the model is unclear, queries become confusing, slow, and error-prone. If the model is clear, queries become almost self-explanatory.

Entities, Relationships, and Schema Design

Entities

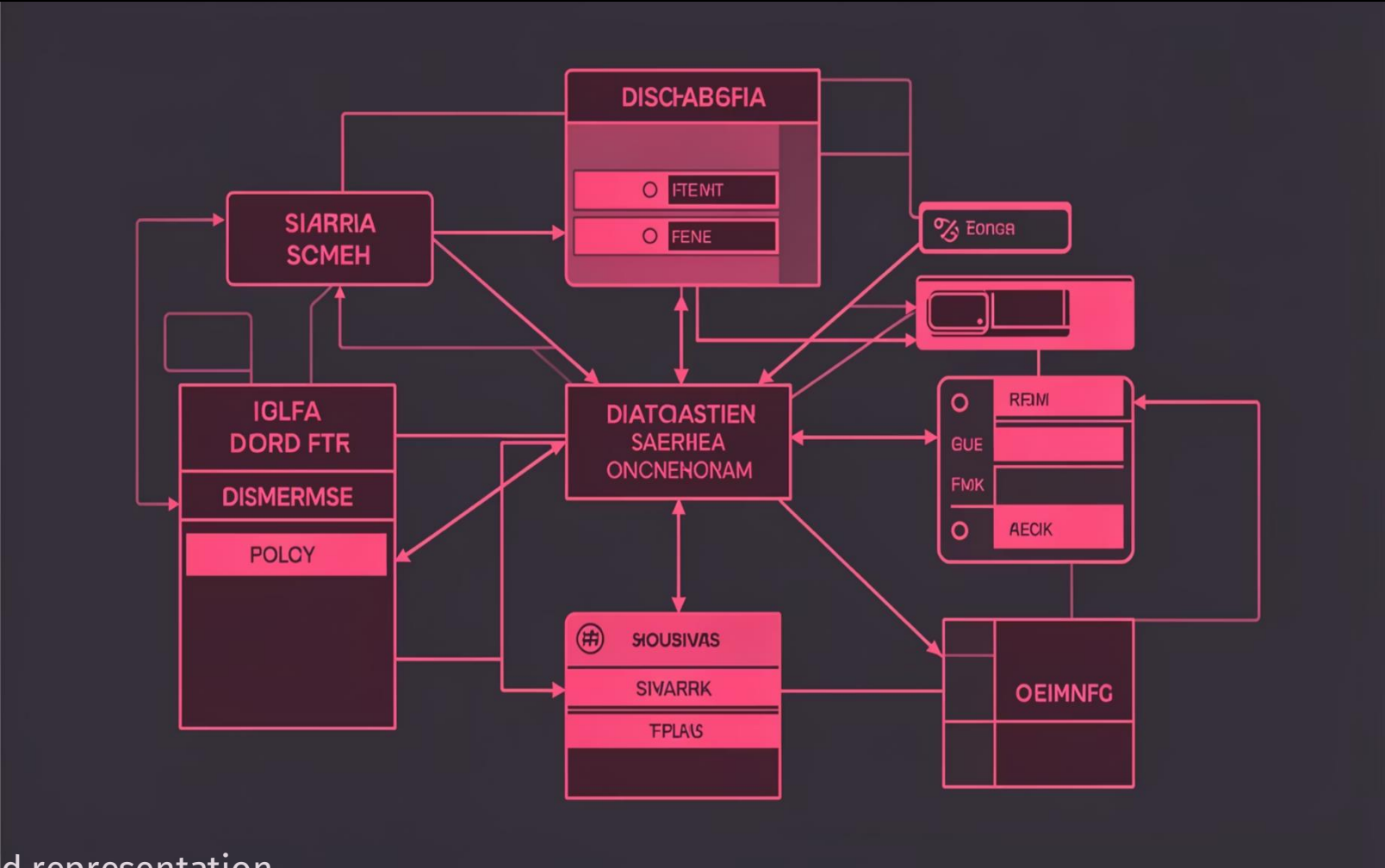
Real-world objects (users, orders, products) stored as attributes in tables or fields in documents.

Relationships

Connections between entities (one-to-one, one-to-many, many-to-many) crucial for data integrity and query complexity.

Schema Design

Structuring entities and relationships into a database-enforced representation, reflecting business rules and protecting data correctness with constraints.



Step 1: Start From the Real World, Not From Tables

Good database modeling always starts with questions like:

- What things exist in this system?
- What do we need to remember about them?
- How do they relate to each other?

For example, in a food delivery system:

- Users place orders
- Orders belong to restaurants
- Orders contain multiple items
- Payments are made for orders

At this stage, we do not think about SQL. We think about **concepts**.

Step 2: Identify Entities

An **entity** represents a distinct concept that has its own identity.

Examples of entities:

- User
- Order
- Restaurant
- Menu Item
- Payment

A useful rule of thumb:

If something has its own lifecycle and can exist independently, it is probably an entity.

Each entity will usually become a table.

Step 3: Define Attributes for Each Entity

Once we identify entities, we decide what information we need to store about each one.

For example:

- User → name, email, created_at
- Order → status, created_at
- Restaurant → name, city
- Payment → amount, method, paid_at

These attributes become columns.

Important idea:

Columns should describe the entity itself, not related entities.

If a column describes something else, it likely belongs in another table.

Step 4: Define Relationships Between Entities

Entities rarely exist in isolation.

We must understand:

- One-to-one relationships
- One-to-many relationships
- Many-to-many relationships

Examples:

- One user → many orders
- One restaurant → many menu items
- One order → many order items

Relationships explain **why JOINS exist**. Without relationships, JOIN has no meaning.

Foreign keys are how relationships are expressed in a relational database.

Step 5: Choose Primary Keys Carefully

Every table needs a primary key.

A primary key:

- Uniquely identifies a row
- Never changes
- Has no business meaning outside identification

Examples:

- user_id
- order_id
- restaurant_id

Primary keys are the anchors of joins and grouping operations later.

Step 6: Normalize the Model First

Before worrying about performance, we normalize.

Normalization ensures:

- Each fact is stored once
- Data is consistent
- Updates are safe

A normalized model gives us:

- Clear table boundaries
- Predictable joins
- Correct aggregation results

This is why normalization comes **before** querying.

Step 7: Think About Questions the Database Must Answer

Now we shift perspective.

Instead of asking:

“How do I write SQL?”

We ask:

“What questions will this system need to answer?”

Examples:

- Show a user's last 10 orders
- Calculate total revenue per restaurant
- Count how many items were sold today

These questions directly map to:

- SELECT → what data
- JOIN → how tables relate
- WHERE → which rows
- GROUP BY → how data is aggregated

If the model is correct, these queries feel natural.

Step 8: How Modeling Affects Querying (Big Picture)

Bad modeling leads to:

- Confusing JOINS
- Incorrect aggregates
- Duplicate rows
- Complex queries

Good modeling leads to:

- Simple SELECT statements
- Meaningful GROUP BY clauses
- Predictable results

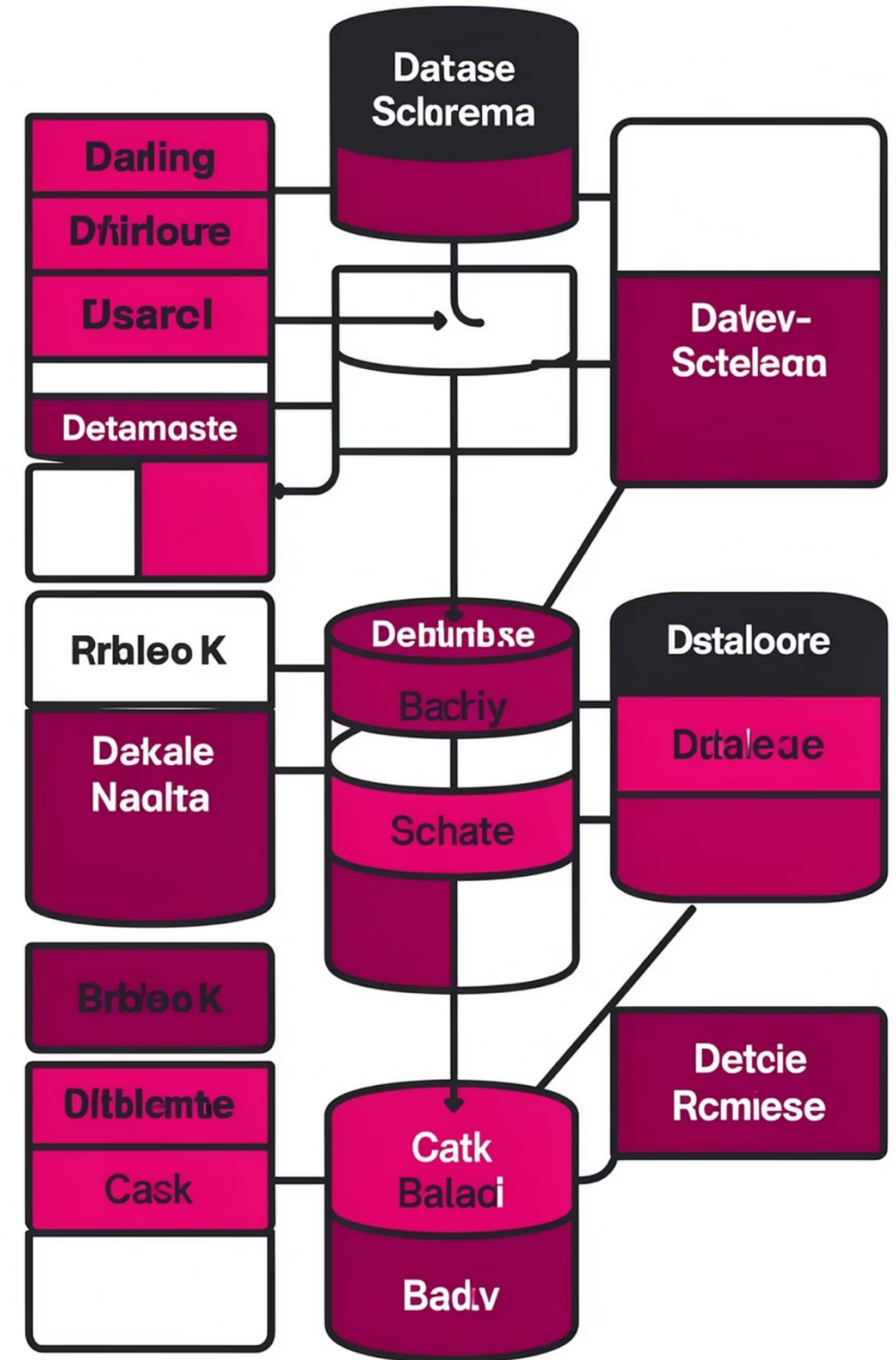
This is the key mental connection students need.

Bridge to SQL Review

Now that we understand how data is modeled,
we can learn how to **ask questions** about that model using SQL.

Section 2

SQL Review: Querying a Well-Modeled Database



1. SELECT: Choosing What You Want to See

SELECT defines what data you want to retrieve. At its simplest, SELECT answers: “Which columns do I want to see in the result?”

Example:

```
SELECT user_id, full_name, email FROM users;
```

This query:

- Reads from a single entity (users)
- Returns one row per user
- Does not combine or summarize data

Key idea for students: SELECT works on rows, not on concepts. The meaning comes from how the table was modeled.

2. WHERE: Filtering Rows

WHERE restricts which rows are included.

Example:

```
SELECT order_id, order_status, created_at FROM orders WHERE user_id = 1;
```

This reads as:

- From all orders
- Keep only the orders that belong to a specific user

Important modeling connection: user_id exists in orders because of a one-to-many relationship

Without that relationship, this query would not be possible

3. JOIN: Following Relationships Between Tables

JOIN is how SQL navigates relationships defined in the data model.

A JOIN answers: “How do these entities relate to each other?”

What this means conceptually:

Example:

```
SELECT o.order_id, r.name AS restaurant_name, o.created_at  
FROM orders o JOIN restaurants r ON r.restaurant_id = o.restaurant_id;
```

- Each order belongs to exactly one restaurant
- That relationship is encoded using a foreign key
- JOIN follows that relationship

JOIN is not magic. It only works because the database was modeled correctly.

INNER JOIN vs LEFT JOIN (Important Distinction)

INNER JOIN

```
SELECT *  
FROM orders o JOIN payments p ON p.order_id  
= o.order_id;
```

Returns:

- Only orders that have payments

LEFT JOIN

```
SELECT *  
FROM orders o LEFT JOIN payments p ON  
p.order_id = o.order_id;
```

Returns:

- All orders
- Payments if they exist
- NULL if they do not

INNER JOIN is used when relationships are mandatory

Modeling insight: LEFT JOIN is often used when relationships are optional

4. GROUP BY: Summarizing Data

GROUP BY is used when we want to aggregate data. Instead of returning one row per record, we return:

- One row per group

Example:

```
SELECT restaurant_id, COUNT(*) AS total_orders FROM orders GROUP BY restaurant_id;
```

This query answers: “How many orders does each restaurant have?”

Important rule: Every column in SELECT must either be aggregated or appear in GROUP BY.

5. Aggregation Functions (The Why Behind GROUP BY)

Common aggregation functions:

- `COUNT()` → how many
- `SUM()` → total
- `AVG()` → average
- `MIN()` / `MAX()` → range

Example:

```
SELECT o.restaurant_id, SUM(p.amount_cents) AS revenue_cents FROM orders o JOIN payments p ON  
p.order_id = o.order_id GROUP BY o.restaurant_id;
```

Modeling connection: Aggregation only makes sense when rows represent events or facts

Poor modeling leads to incorrect totals and double counting

6. GROUP BY + JOIN (Very Common Pattern)

This is where students usually struggle, so clarity matters.

Example:

```
SELECT r.name AS restaurant_name, COUNT(o.order_id) AS total_orders FROM restaurants r LEFT JOIN orders  
o ON o.restaurant_id = r.restaurant_id GROUP BY r.restaurant_id, r.name;
```

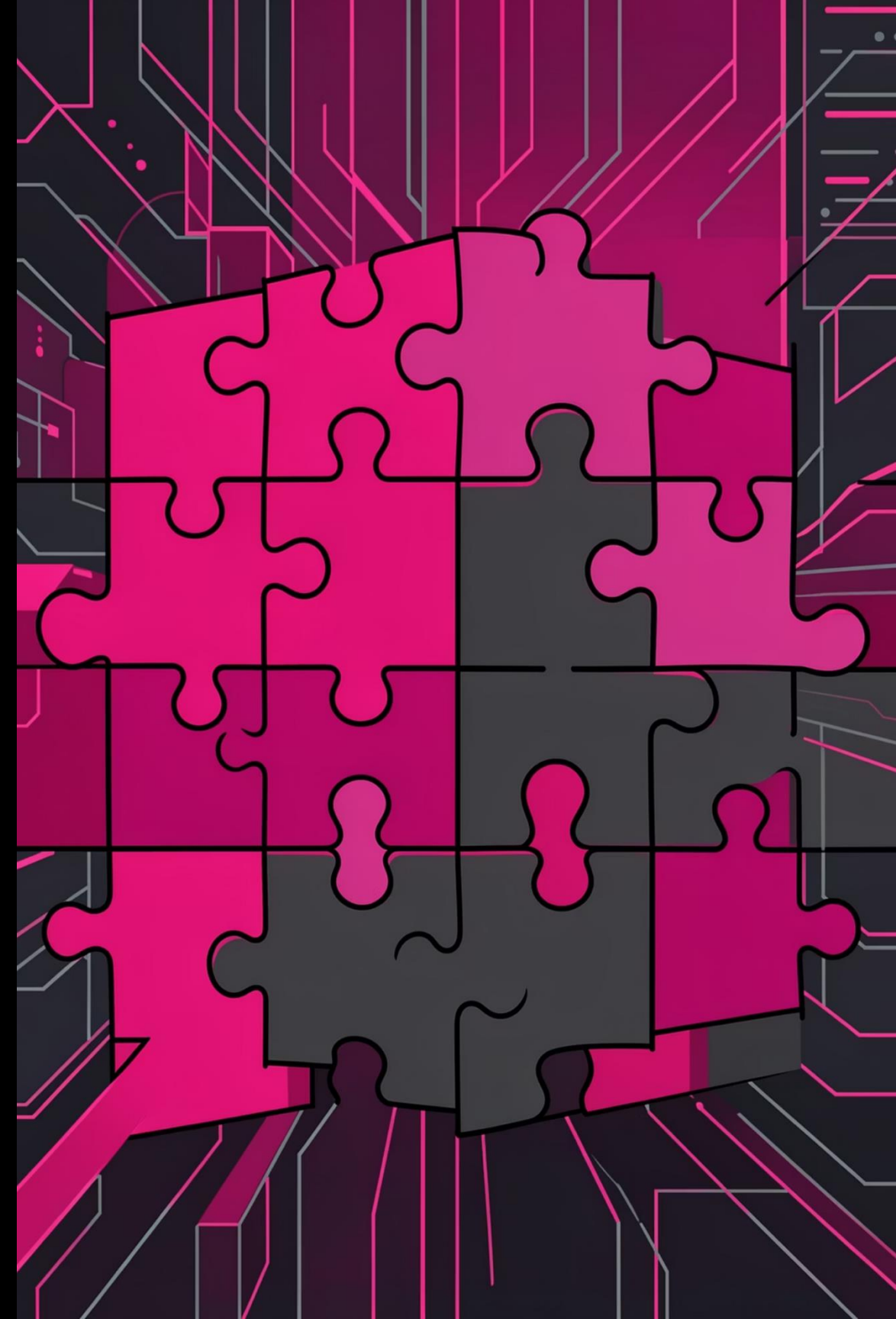
Conceptually:

- One restaurant → many orders
- We group by the restaurant
- We count related orders

Key insight: JOIN expands rows **GROUP BY** collapses them again in a meaningful way

7. Common Mistakes

- Grouping by the wrong column → wrong totals
- Forgetting JOIN conditions → Cartesian products
- Aggregating denormalized data incorrectly → double counting
- Using GROUP BY to “fix” a bad model → hides design problems instead of solving them



Section 3

8. How Modeling Makes Queries Easier

Well-modeled database:

- Simple JOIN conditions
- Predictable GROUP BY logic
- Clean SELECT statements

Poorly modeled database:

- Confusing queries
- Unexpected duplicates
- Hard-to-debug results

Key takeaway: SQL does not fix modeling mistakes. It only exposes them.

Now that we understand how to query a well-modeled database, we can look at how schema design choices affect query performance and complexity.

Normalization in Database Design

Normalization organizes data to minimize redundancy and dependency, ensuring each fact is stored in one place. Its primary goal is correctness, clarity, and maintainability, especially in transactional systems.

Benefits:

- Avoid duplicate/conflicting data
- Safe updates
- Clear table representation
- Enforce business rules

Core Idea: One Fact, One Place

Normalization ensures each table represents one concept, and each column describes only that concept. This prevents common problems like data duplication and inconsistency.



Users Table

Describes users.



Orders Table

Describes orders.



Order Items Table

Describes order-item relationships.

Functional Dependency

If one value uniquely determines another, they belong to the same table. This prevents columns from depending on non-primary key attributes.

The Three Normal Forms

01

First Normal Form (1NF)

Each column holds a single value; no repeating groups or lists. Data is structured, not text blobs.

02

Second Normal Form (2NF)

Must be in 1NF. Non-key columns depend on the whole primary key, not just part of it (for composite keys).

03

Third Normal Form (3NF)

Must be in 2NF. Non-key columns do not depend on other non-key columns, preventing indirect dependencies.

These forms ensure data integrity, prevent anomalies, and make schema evolution safer over time.

First Normal Form (1NF) Examples

Card about First Normal Form (1NF) with concrete examples. Include the bad example of items column with "Burger, Fries, Soda" and the good example of order_items table. Make it clear and practical.

Bad Example: Violating 1NF

Consider an order table where the "items" column contains multiple values, such as "Burger, Fries, Soda". This violates 1NF because each cell should hold only a single, atomic value.

Burger, Fries, Soda

Good Example: Adhering to 1NF

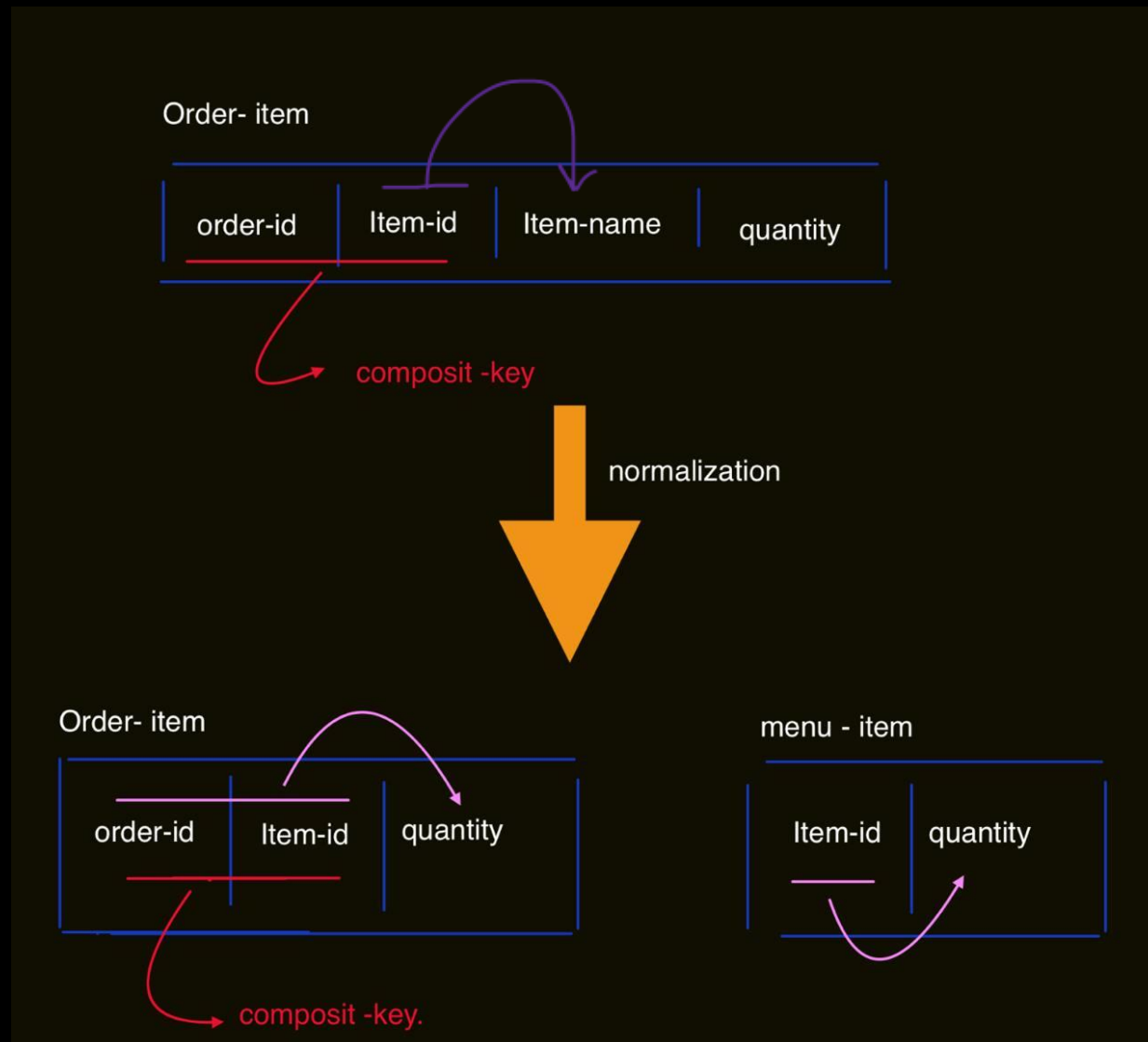
To fix this, we create a separate `order_items` table. Each row in this new table represents a single item for a given order, ensuring atomic values and proper data structure.

101	1	Burger	1
102	1	Fries	1
103	1	Soda	1
104	2	Pizza	1
105	2	Soda	2

Second Normal Form (2NF)

The Second Normal Form (2NF) requires that a table must first be in 1NF, and all non-key attributes must be fully dependent on the entire primary key. This is particularly relevant when dealing with composite primary keys.

Problem: Violating 2NF with a Composite Key



order_id	item_id	item_name	quantity
1	1	Burger	1
1	2	Fries	1
2	1	Burger	2

order_id	item_id	quantity
1	1	1
1	2	1
2	1	2

order_items

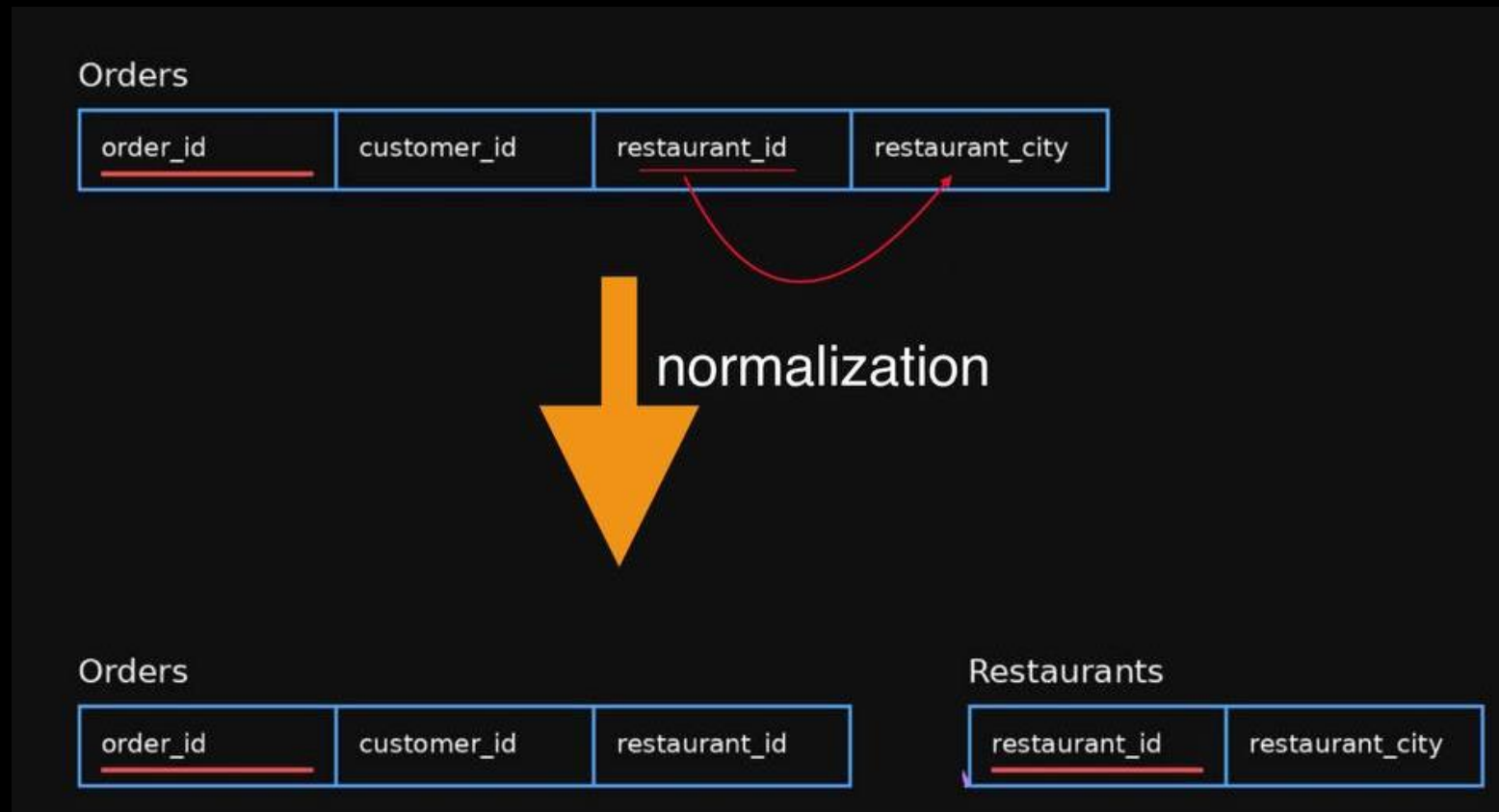
item_id	item_name
1	Burger
2	Fries

menu_items

Third Normal Form (3NF)

The Third Normal Form (3NF) requires a table to be in 2NF, and all non-key attributes must be non-transitively dependent on the primary key. This means that non-key attributes should not depend on other non-key attributes.

Problem: Violating 3NF with Transitive Dependency



order_id	customer_id	restaurant_id	restaurant_city
101	C001	R001	New York
102	C002	R002	Los Angeles
103	C001	R001	New York
104	C003	R003	Chicago

order_id	customer_id	restaurant_id
101	C001	R001
102	C002	R002
103	C001	R001
104	C003	R003

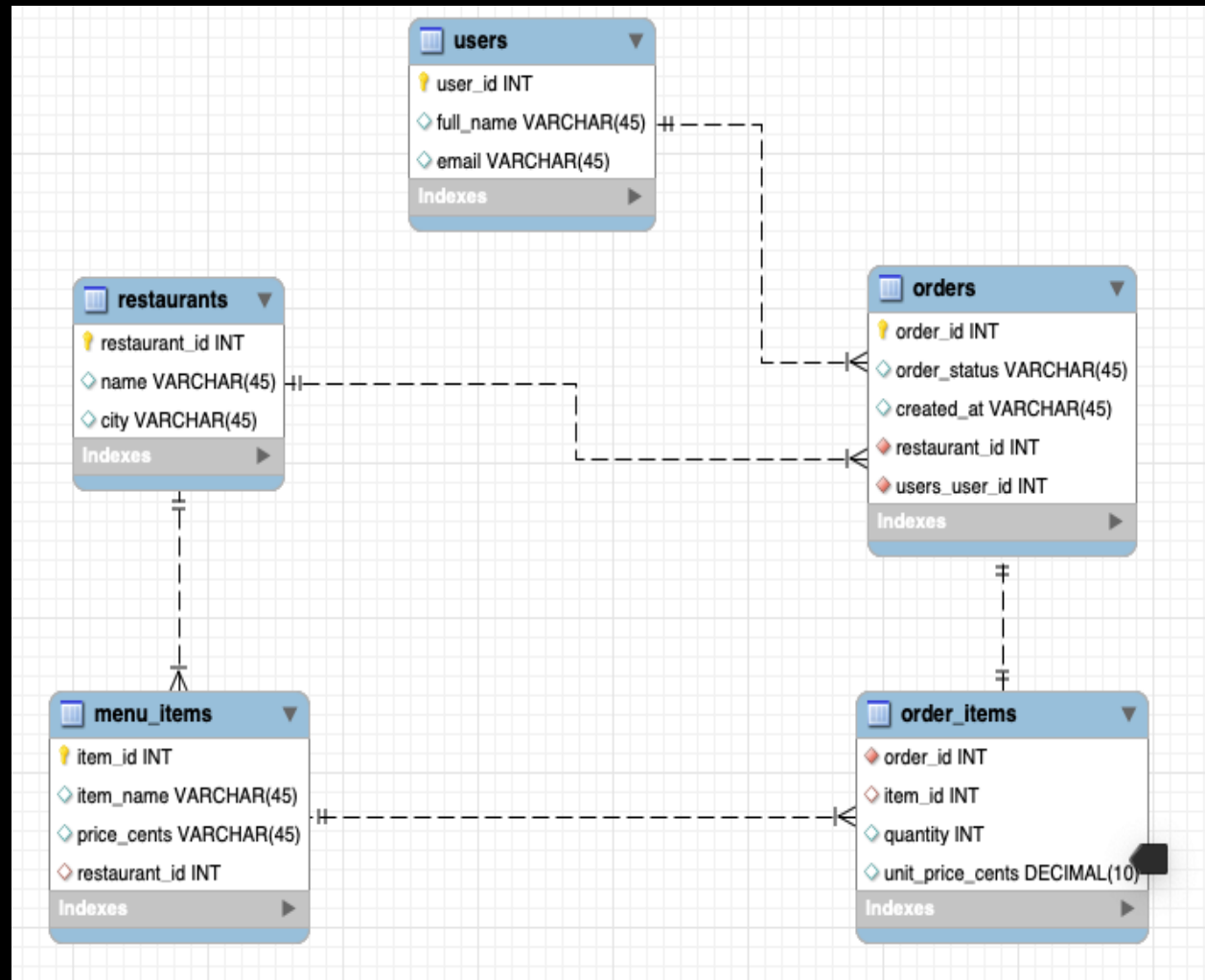
orders table

restaurant_id	restaurant_city
R001	New York
R002	Los Angeles
R003	Chicago

restaurants table

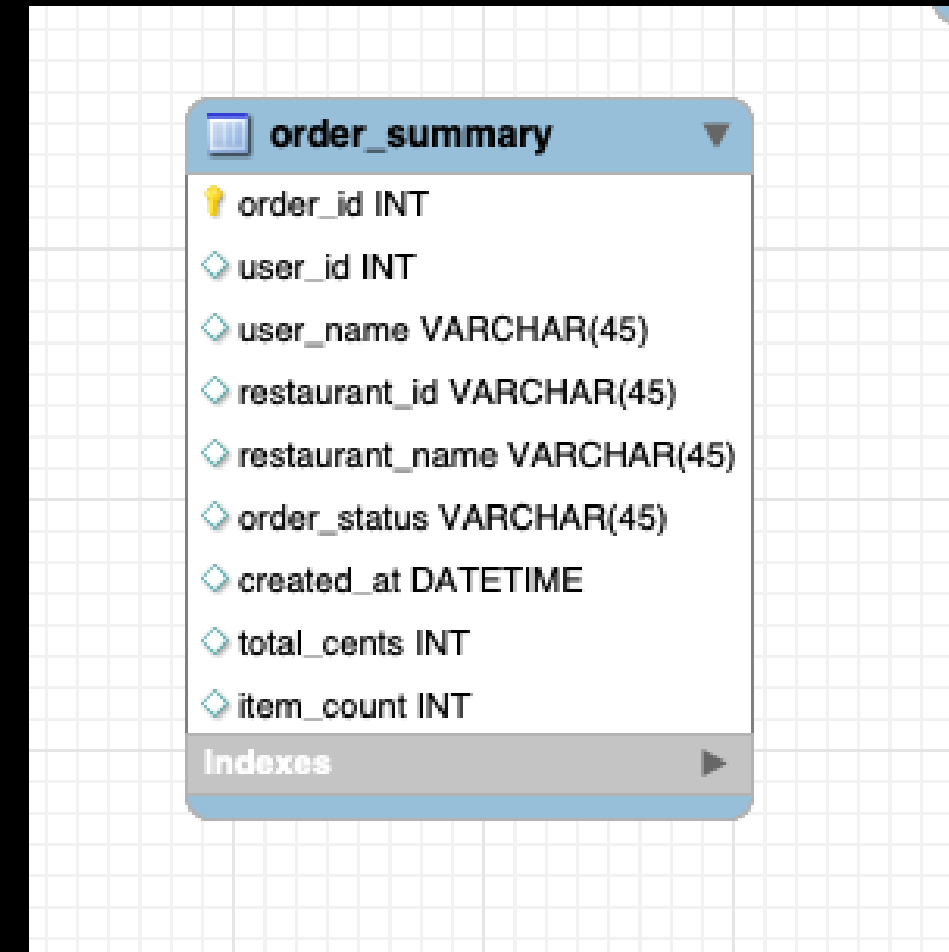
Normalization vs. Denormalization

Normalized Model (Source of Truth)



Separates data to preserve truth, minimizing redundancy. Requires joins for queries, ensuring strong consistency.

Denormalized Read Model



Groups data to serve questions faster, intentionally introducing redundancy. Minimizes joins for faster reads, sacrificing some consistency.

Insight: Normalization separates data to preserve truth. Denormalization groups data to serve questions faster.

SQL Queries: Normalized vs. Denormalized

Normalized Design: Complex Queries

In a normalized design, retrieving comprehensive order details often requires joining multiple tables. This ensures data integrity and minimizes redundancy but can lead to more complex and resource-intensive queries, as shown in this example to get full order details.

```
SELECT
    o.order_id, o.customer_id, mi.item_name
    oi.quantity,  r.restaurant_city
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN menu_items mi ON oi.item_id = mi.item_id
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
WHERE  o.order_id = 101;
```

Denormalized Design: Simple Queries

A denormalized design, such as an `order_summary` table, intentionally introduces redundancy to pre-aggregate data or combine information from related tables. This allows for much simpler and faster queries, as all the necessary information for a common analytical question is often available in a single table.

```
SELECT
    order_id, customer_id, item_name,
    quantity, restaurant_city
FROM. order_summary
WHERE  order_id = 101;
```

`order_summary` table schema:

order_id	customer_id	item_name	quantity	restaurant_city
101	C001	Burger	1	New York
101	C001	Fries	1	New York

Denormalization Examples

We'll now look at three practical examples of denormalization, illustrating how intentional redundancy can optimize database performance for specific use cases. We will explain why each approach is useful and when to consider implementing them.

1. Order Summary Table (Denormalized Fields)

An `order_summary` table combines frequently accessed information from various normalized tables (e.g., customer details, item names, restaurant locations) into a single, wider table. This eliminates the need for complex joins during read operations, significantly speeding up common queries, especially for reporting and analytics.

Use Case: Ideal for analytical queries or read-heavy applications where rapid access to aggregated or combined data is crucial, and the source data changes less frequently.

order_id	customer_name	item_name	quantity	restaurant_city
101	Alice	Burger	1	New York
101	Alice	Fries	1	New York
102	Bob	Pizza	1	Los Angeles

`order_summary`

Denormalization Examples

We'll now look at three practical examples of denormalization, illustrating how intentional redundancy can optimize database performance for specific use cases. We will explain why each approach is useful and when to consider implementing them.

2. Duplicating Restaurant Name in Orders Table

Instead of relying solely on a foreign key to a separate `restaurants` table for the restaurant name, you might duplicate the `restaurant_name` directly into the `orders` table. This is technically redundant, as the name already exists in the `restaurants` table, but it can avoid a join for simple order listings.

Use Case: Useful when you frequently need the restaurant name alongside order details and the restaurant name changes very rarely. The overhead of updating duplicated names must be less than the performance gain from avoiding joins.

order_id	customer_id	restaurant_id	restaurant_name
101	C001	R001	Burger Joint
102	C002	R002	Pizza Place
103	C001	R001	Burger Joint

`orders` table with duplicated `restaurant_name`

Denormalization Examples

We'll now look at three practical examples of denormalization, illustrating how intentional redundancy can optimize database performance for specific use cases. We will explain why each approach is useful and when to consider implementing them.

3. Pre-computed Aggregates (total_cents, item_count)

Adding columns like `total_cents` (total order cost) or `item_count` (total items in an order) directly to the `orders` table. These values can be derived from other tables (e.g., `order_items`) but are stored directly to avoid on-the-fly calculations every time the order details are fetched.

Use Case: Highly effective for frequently accessed aggregate values, dashboards, or e-commerce carts where immediate display of totals is critical. Requires careful management to ensure these pre-computed values are updated whenever underlying data changes.

order_id	customer_id	total_cents	item_count
101	C001	1500	2
102	C002	2500	1
103	C001	750	1

`orders` table with pre-computed aggregates

Exercise: What Would You Denormalize and Why?

Imagine you are designing a database for a food delivery system. We want to create an `order_summary` table that can quickly provide high-level information about an order without complex joins. Below is a list of potential fields. For each, decide whether it would be a good candidate for denormalization into the `order_summary` table and provide your reasoning.

- A fast “My Orders” page
- A restaurant dashboard
- A mobile API with low latency

You already have a fully normalized schema

Exercise: What Would You Denormalize and Why?

- user_name
- user_email
- restaurant_name
- restaurant_city
- total_price
- item_count
- order_status
- payment_method

User Name

Candidate: Good

Reasoning: User details like name and email are frequently accessed when viewing order history or customer service. They typically change infrequently. Denormalizing avoids a join to the `users` table, speeding up reads for common order summary displays.

Restaurant Name

Candidate: Good

Reasoning: Restaurant name and location are crucial for an order summary and are often displayed together. This data changes rarely. Denormalizing these fields eliminates a join to the `restaurants` table, enhancing query performance for order listings.

Total Price

Candidate: Good

Reasoning: The total price is a pre-computed aggregate that would otherwise require summing line items. It is critical for quick display in order summaries and dashboards. Denormalizing saves on-the-fly calculations, provided it's updated consistently when underlying items change.

Item Count

Candidate: Good

Reasoning: Similar to total price, item count is a pre-computed aggregate frequently needed for order summaries. Denormalizing avoids a join and count operation on the `order_items` table, improving performance for summary views, with similar update considerations as total price.

Order Status

Candidate: Good

Reasoning: Order status (e.g., "pending," "delivered") is dynamic but essential for immediate user feedback. While it changes, it's typically a direct value for the current state. Denormalizing allows quick filtering and display without joining to a separate status or history table.

Exercise: What Would You Denormalize and Why? Cont.

- user_name



User Email

Candidate: Bad

- user_email

Reasoning: User details like name and email are frequently accessed when viewing order history or customer service. They typically change infrequently. Denormalizing avoids a join to the `users` table, speeding up reads for common order summary displays.

- restaurant_name



Restaurant City

Candidate: Bad

- restaurant_city

- total_price

Reasoning: Restaurant name and location are crucial for an order summary and are often displayed together. This data changes rarely. Denormalizing these fields eliminates a join to the `restaurants` table, enhancing query performance for order listings.

- item_count



Payment Method

Candidate: Consider Carefully

- order_status

- payment_method

Reasoning: The specific payment method (e.g., "Visa," "PayPal") might be useful for summary reporting. It generally changes only if the user updates it for future orders, not for a past order itself. While denormalizing avoids a join to a `payments` table, ensure no sensitive data is stored here and that performance gains outweigh the slight increase in redundancy and potential maintenance if payment method details are complex or change frequently.

Denormalization in Real-World Systems

Denormalization intentionally introduces redundancy to improve performance, simplify queries, or support specific access patterns. It's a conscious choice, typically applied after a correct normalized model exists and performance issues arise.

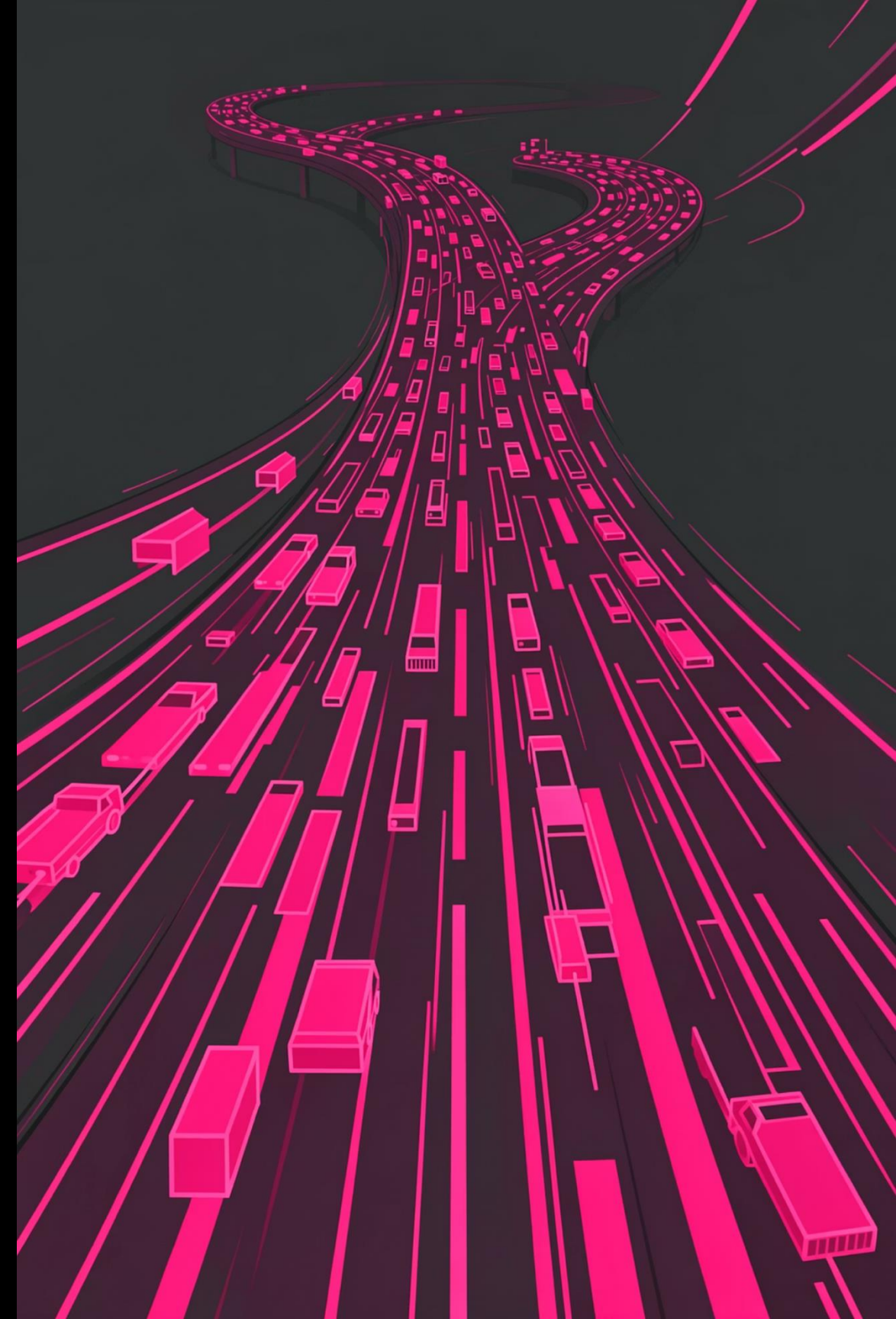
The Core Trade-Off

Gain:

- Faster read queries
- Simpler SQL
- Fewer joins

Give Up:

- Single source of truth
- Simpler writes
- Automatic consistency



Designing for Read-Heavy vs. Write-Heavy Systems

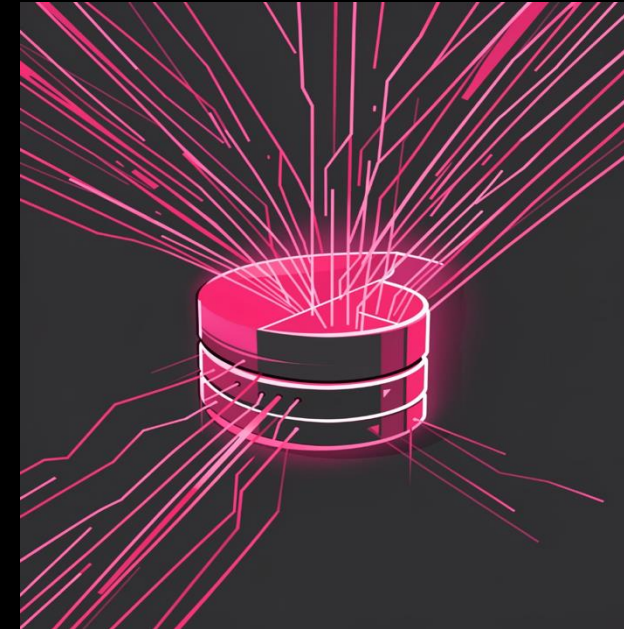
Read-Heavy Systems



Data changes infrequently, read many times, low latency critical.
Examples: "My Orders" pages, restaurant dashboards, user profiles.

- Prioritize denormalized tables
- Precomputed aggregates
- Optimize for fast reads

Write-Heavy Systems



Constant inserts, write throughput matters more, strong consistency critical. Examples: Logging systems, payment systems, event ingestion.

- Simple append-only tables
- Minimal indexing/constraints
- Fully normalized schema (payments)

Read-Heavy vs. Write-Heavy Systems

Read-Heavy Systems

Read-heavy systems are characterized by a significantly higher volume of read operations compared to write operations. These systems prioritize fast data retrieval and often leverage denormalization to achieve this.

Examples:

- My Orders page: Users frequently check past and current orders, but rarely modify them.
- Restaurant Dashboard: Restaurant owners constantly view sales, inventory, and customer data, with updates being less frequent.
- User Profiles: Profile information is viewed by many, but updated infrequently by the user themselves.

Design Choices:

- Denormalization: Pre-joining data or creating summary tables to avoid complex joins on read.
- Caching: Storing frequently accessed data in memory for rapid retrieval.
- Indexing: Heavily indexing columns used in search and filter operations.
- Read Replicas: Distributing read load across multiple database instances.

Read-Heavy vs. Write-Heavy Systems: Cont.

Write-Heavy Systems

Write-heavy systems are dominated by a large volume of data insertion and modification operations.

These systems prioritize data integrity, atomicity, and throughput for writes.

Examples:

- Logging Systems: Continuously ingesting high volumes of event logs.
- Payment Systems: Processing numerous financial transactions, where data integrity is paramount.
- Event Ingestion: Receiving and storing real-time data streams from various sources.

Design Choices:

- Normalization: Maintaining data integrity and minimizing redundancy to prevent write anomalies.
- Sharding/Partitioning: Distributing write load across multiple servers to improve scalability.
- Asynchronous Processing: Decoupling write operations from user requests to improve response times.
- Append-only Logs: Optimizing for sequential writes, often seen in journaling and logging.

Key Trade-off: Read-heavy systems prioritize query performance through redundancy, while write-heavy systems prioritize data integrity and write throughput through structured data models and distributed writes.

Key Takeaways

Data modeling is an evolving process. Strong models are adaptable, understandable, and aligned with how data is used.

Normalization for Truth

Design for correctness, consistency, and maintainability first.

Denormalization for Speed

Optimize for performance and query simplicity when needed.

Context is Key

Understand dominant workloads (read-heavy vs. write-heavy) to make informed design choices.





THANK YOU

 shiftkeylabs.ca

 [@ShiftKeyLabs](https://twitter.com/ShiftKeyLabs)

 info@shiftkeylabs.ca



We want to hear from you!

Scan the QR Code to provide your feedback

